

Clasificación Interpretable de Flujos IoT: Random Forest con Análisis SHAP sobre RT-IoT2022

Archibald Emmanuel Carrion Claeys

PPCI, Universidad de Costa Rica

1 de julio de 2026

- Los dispositivos IoT crecen rápidamente en hogares y entornos industriales.
- Su superficie de ataque de red crece proporcionalmente.
- Detección de intrusiones (IDS) en tráfico IoT → problema de **clasificación multiclase supervisada**.
- Dataset: **RT-IoT2022** (UCI ID 942)
 - Tráfico *real*, capturado en un banco de pruebas físico.
 - 81 features de nivel de flujo (tasas de paquetes, tamaños de payload, flags TCP, etc.).
 - 12 clases: ataques (DoS, escaneos NMAP, fuerza bruta) y tráfico benigno (MQTT, ThingSpeak, foco Wipro).

- Los modelos más avanzados suelen ser **cajas negras** (redes neuronales, ensambles complejos).
- Un analista de seguridad necesita saber *por qué* un flujo fue clasificado como ataque.
- RT-IoT2022 tiene **fuerte desbalance de clases**.
- **Pregunta central:** ¿Podemos lograr un modelo preciso y interpretable, sin que el desbalance de clases distorsione la explicación?

Motivación personal: trabajar con modelos que un analista humano realmente pueda auditar, no solo un número de accuracy.

Referencia	Dataset	Modelo(s)	SHAP
Sharmila & Nagapadma (2023)	RT-IoT2022	Autoencoder	×
Airlangga (2025)	RT-IoT2022	RF, GB, LR, MLP	×
Sama et al. (2024)	RT-IoT2022	KNN, GB, XGB, RF, DT	×
Tuteja et al. (2025)	RT-IoT2022	RF (+SMOTE)	×
Sourav et al. (2025)	RT-IoT2022	PSO+CNN+XGBoost	✓
Este trabajo	RT-IoT2022	RF	✓

¿Qué es original aquí?

- Primer trabajo conocido que aplica **RF + SHAP directamente** sobre RT-IoT2022 (sin capa intermedia tipo CNN).
- Técnica propia de **muestreo y normalización balanceada por clase** para que SHAP no sea dominado por las clases mayoritarias.
- Visualización propia (*dot summary plot*) diseñada para 12 clases muy desbalanceadas.

Implementación: Preprocesamiento

- Descarga vía `ucimlrepo` (ID 942).
- **One-hot encoding** de `proto` y `service` → 13 columnas binarias.
- 81 columnas numéricas sin transformar → **94 features** finales.
- Codificación de etiquetas con `LabelEncoder`.
- División **80/20 estratificada** (`random_state=42`) para preservar la proporción de clases.

*Este proyecto es clasificación **asincrónica** / **offline**: se entrena y evalúa sobre datos ya capturados, no sobre tráfico en vivo.*

Implementación: Modelo

- RandomForestClassifier con 100 árboles.
- `class_weight='balanced'` para compensar el desbalance durante el entrenamiento.
- `n_jobs=-1` para paralelizar la construcción de árboles.
- Evaluación: *accuracy*, **F1 macro** (pondera cada clase por igual, ideal para datasets desbalanceados) y matriz de confusión.

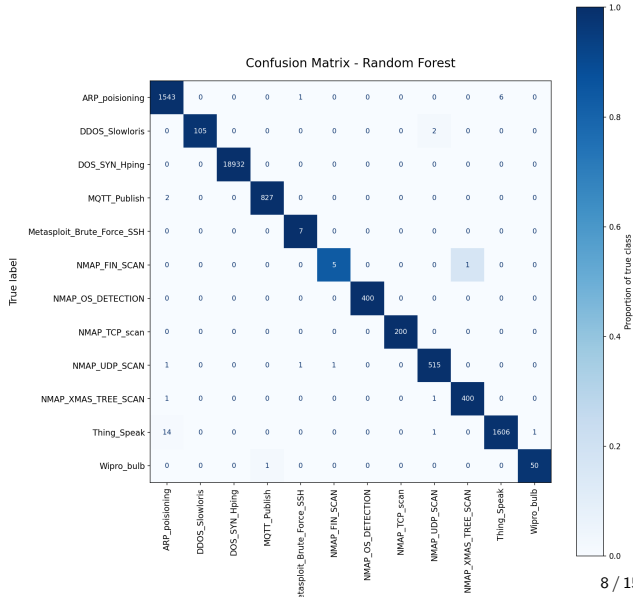
Implementación: Pipeline SHAP

- ① `shap.TreeExplainer` sobre el Random Forest entrenado.
- ② **Muestreo balanceado por clase:** hasta 100 muestras por clase (en vez de una muestra proporcional al dataset).
 - Sin esto, clases minoritarias casi no aportan señal al promedio de $|\text{SHAP}|$.
- ③ **Normalización dentro de cada clase:** los aportes de cada feature se expresan como % del total de esa clase (igual filosofía que la matriz de confusión normalizada por fila).
- ④ Dos visualizaciones: gráfico de barras apiladas por clase, y *dot summary plot* propio (un punto = una clase, por feature).

Resultados: Desempeño de Clasificación

- **Accuracy:** 99.86 %
- **F1 macro:** 0.971
- 24,624 muestras de prueba

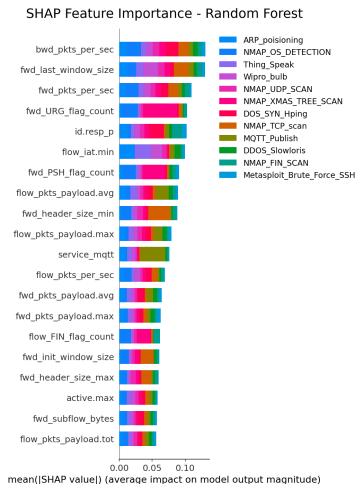
Clase	F1
DOS_SYN_Hping	1.00
MQTT_Publish	1.00
Metasploit_BF_SSH	0.88
NMAP_FIN_SCAN	0.83
Wipro_bulb	0.98



Resultados: Por qué normalizar la matriz de confusión

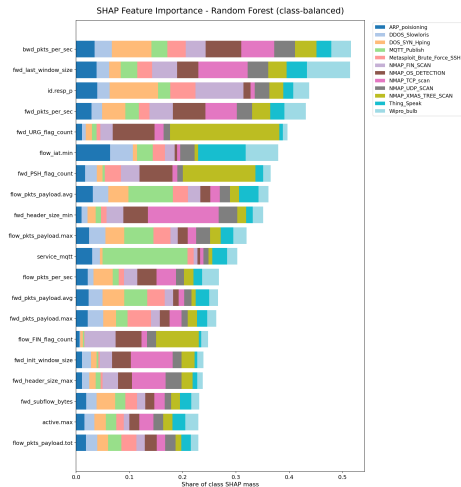
- Color = proporción respecto al total de la clase verdadera (fila).
- Número mostrado = conteo real (no oculta información).
- Sin esto, DOS_SYN_Hping (18,932 muestras) saturaría el mapa de color y los errores en clases minoritarias serían invisibles.
- Principal fuente de error: Thing_Speak → 14 falsos negativos como ARP_poisoning.

Resultados: Importancia SHAP – Sin Normalizar



El eje X mezcla magnitudes de clases con tamaños muy distintos:
DOS_SYN_Hping domina casi todas las barras.

Resultados: Importancia SHAP – Balanceada por Clase

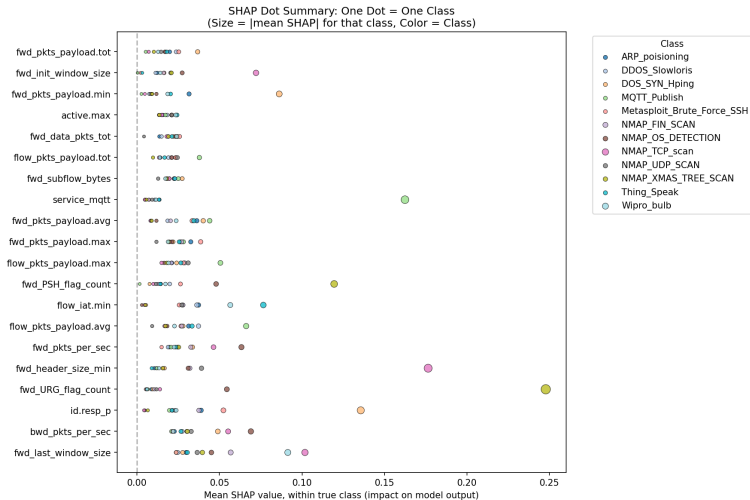


Muestreo balanceado (100/clase) + normalización → cada clase aporta una porción visible y comparable.

Resultados: Patrones Específicos por Clase

- `fwd_URG_flag_count` → casi exclusivamente `NMAP_XMAS_TREE_SCAN` (coincide con que este escaneo activa la bandera URG).
- `service_mqtt` → dominado por `MQTT_Publish`.
- `fwd_header_size_min` / `fwd_init_window_size` → explicados por `NMAP_TCP_scan`.
- Ninguna clase “gana” todas las barras: el modelo usa firmas de features distintas por tipo de ataque.

Resultados: Visualización Propia – Dot Summary Plot



Resultados: Lectura del Dot Summary Plot

- Un punto = una clase, para cada feature (12 puntos por fila).
- Posición en X = SHAP promedio (empuja hacia / en contra de la clase).
- Alternativa compacta al *beeswarm* tradicional, que se vuelve ilegible con 12 clases y cientos de puntos por feature.

- Random Forest logra **99.86 % accuracy** y **0.971 F1 macro** en RT-IoT2022, manteniéndose totalmente interpretable.
- SHAP aplicado **directamente** sobre las 94 features originales, sin transformación intermedia tipo CNN.
- El **desbalance de clases exige normalización**: tanto en la matriz de confusión como en SHAP, de lo contrario las clases minoritarias quedan invisibles.
- Próximos pasos: despliegue en tiempo real (edge, P4/Mininet), validación cruzada con Bot-IoT/CICIoT2023, robustez adversarial.

¡Gracias! Preguntas.